CLASHBOT: A Knowledge-Grounded LLM Agent That Plays Clash Royale in Real Time from Pixels

David Tyler Jackson (5298477)
CAP 6938 AI Knowledge Systems, University of Central Florida
Instructor: Dr. Ser Nam Lim

***Abstract*—We present CLASHBOT, a non-embedded agent that plays the real-time card game Clash Royale on an iPhone through macOS screen mirroring, with pixels in and taps out and every model running on one laptop. A fast loop runs YOLOv8 detection, OCR and tracking, a learned play/wait gate, and a candidate generator built from a tactical lane simulator, hybrid retrieval over 10,923 states from 197 pro games, and a knowledge graph of 840 nodes and 27,112 edges. A 600M-parameter decision LLM, fine-tuned on the pro games, chooses one of at most four candidates and remains the decision maker. On held-out games it picks the pro's play 50.7% of the time (95% CI 47.2–54.0) among four hard options, against 25% chance and 38.4% for a placement prior, but a tuned LightGBM ranker (48.2%) is statistically indistinguishable, and several course techniques do not beat simple baselines; we report these results as measured. Live, a decision takes 339 ms from capture to click at the median, and the agent won its first Training Camp battle; in a trial of [TBD: 10] battles it won [TBD: k].**

***Index Terms*—game agents, knowledge graphs, retrieval-augmented decision making, LLM ranking, real-time systems**

I. INTRODUCTION

Clash Royale is a real-time, two-player card game played on phones. Each player brings a hidden deck of eight cards, holds four of them, and spends elixir, which regenerates up to ten, to place troops, buildings and spells on an 18×32-tile arena; a match lasts three minutes plus up to two minutes of overtime [1]. CLASHBOT is a *non-embedded* agent [1]: it has no game API, reads the screen of an iPhone mirrored onto a Mac, and acts by clicking the mirrored window, the interface a human uses.

The task is hard for five measurable reasons. (i) *Real time*: perception runs on a stream with a 10 fps target (7.97 fps reached in our latest run) and the whole capture-to-click pipeline takes 338.9 ms at the median. (ii) *Partial, noisy observation*: the opponent's deck, hand and elixir are hidden, and our own reading of the board is imperfect; the pro's recorded move passes our perceived-legality check on only 0.542–0.597 of states. (iii) *A structured action space plus a timing decision*: an action is card × lane × zone or hold (D-010), and pros deploy within the next second on about a fifth of seconds (dense act rate 0.204). (iv) *Sparse supervision*: the expert data are 197 pro games (about 12 match-hours, one creator) with no reward and no simulator, and the Training Camp opponent plays level-15 cards. (v) *Consumer hardware*: everything runs on an Apple M5 laptop with 32 GB and no paid API.

The design follows the course's retrieve-then-rank and agent-with-tools patterns [2]–[4]. Deterministic tools (a tactical engine with a lane simulator, hybrid retrieval over pro states, and a knowledge graph) propose at most four candidate plays with short text notes; a decision LLM fine-tuned on the pro games picks one. By project decision the LLM stays the decision maker and no tool picks the move (D-045; design decisions are logged in the repository as D-001 to D-053). Our contributions:

1) A non-embedded, two-speed agent that plays live through iPhone Mirroring with all models local (Sec. III).
2) A knowledge layer: retrieval over 10,923 pro states (644-d structured vector and BM25, fused by RRF), a two-version knowledge graph with lane-simulation edges and 3,620 mined pro answers, and a tactical engine, all rendered as context for a fine-tuned setwise LLM ranker.
3) A leakage-checked benchmark suite. Our first multiple-choice options leaked the answer (a rule reading only option keys scored 0.852 on test); we rebuilt them as a 2×2 card×placement grid and compare ten rankers with cluster-bootstrap CIs, plus ablations for retrieval, KG embeddings, mined rules, timing, next-card prediction and knowledge editing, negative results included (Sec. V).
4) Live evaluation in Training Camp: 13 recorded runs with 931 logged decisions, a first win on 2026-11-07, and a trial of [TBD: 10] battles compared with KataCR's 1/20 (Sec. V-I).
5) Demo tooling: a launcher that taps only when the opponent banner reads "Trainer", behind a two-switch live guard, and a decision dashboard (Fig. 2).

The bot plays only Training Camp and friendly battles, never ranked matches against people (D-031, D-050).

II. RELATED WORK

Game agents. AlphaStar [5] and OpenAI Five [6] reached expert level through game APIs and large-scale self-play, which a phone game without an API does not offer. KataCR [1] is the closest work: a non-embedded Clash Royale agent with YOLOv8 detectors trained on synthetic composites, OCR, and an offline-RL StARformer policy in the Decision Transformer formulation [7], trained on 105 episodes (113,981 frames) of one human against the built-in AI. Over 20 Training Camp games per checkpoint, its headline model won one (5%)

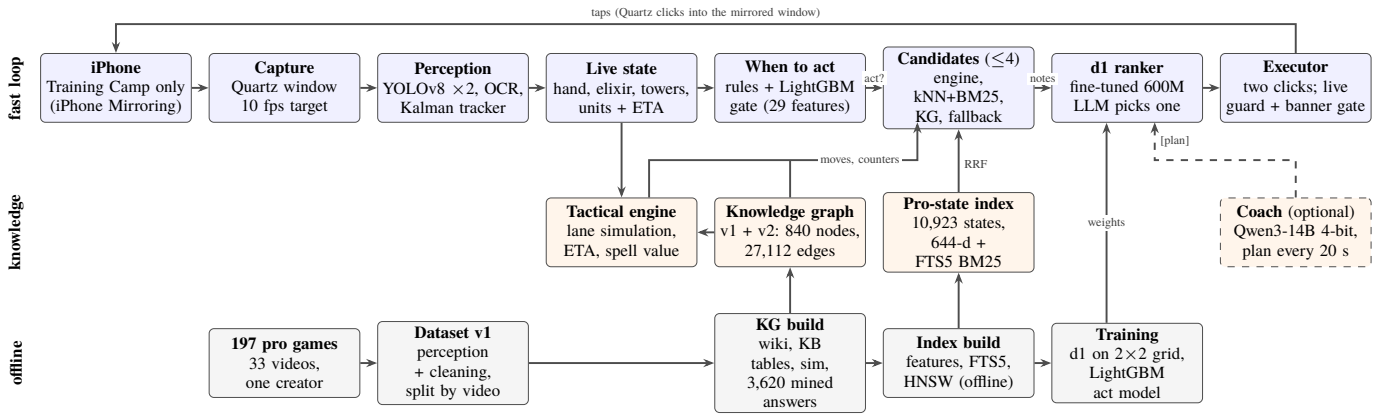


Fig. 1. Two-speed architecture (D-007). The fast loop runs on every decision (338.9 ms p50 capture to click); tools in the knowledge layer only propose candidates and write notes, the fine-tuned d1 LLM picks the move (D-045). The coach is optional and off by default. Every decision is logged and can be replayed in the dashboard (Fig. 2).

and nine models ranged from 0 to 10%. We reuse its published detector weights [8] and its Training Camp protocol. Claude Royale [9] puts an LLM in the loop: Claude Haiku sub-agents read screenshots and click through BlueStacks at about 7 s per decision per agent, on the human ladder (1,723 trophies reported), so it has no built-in-AI win rate to compare with.

LLM agents and ranking. ReAct interleaves reasoning with tool calls [10], Reflexion adds verbal self-feedback [11] and LATS adds tree search [12]; each step costs seconds. A real-time budget rules out generation in the fast loop, so our LLM acts as a setwise reranker [13], [14] over tool proposals, as in two-stage recommender serving, where a cheap stage retrieves and an expensive stage ranks [3], [15]. Fine-tuning an LM to choose among items follows TALLRec and P5 [16], [17]; putting graph facts in the context follows GraphRAG [18]; deciding when to consult the model follows adaptive retrieval [19], [20].

III. System

A. Overview

Fig. 1 shows the two-speed design (D-007). The fast loop runs capture, perception, a clean live state, a play/wait gate, candidate generation, the d1 ranker and the executor. An optional slow loop (Qwen3-14B, 4-bit [21]) rewrites a three-line plan every 20 game seconds in quiet moments; it never picks a move and was used in one live run (21 plans, 5.98/6.86 s p50/p95). The default live path calls no generative LLM.

B. Perception and live state

Frames come from the iPhone Mirroring window by Quartz window capture (632×1392 pixels, 22–38 ms per grab). Two YOLOv8 detectors [22] with KataCR's published weights [8] (87.8 MB each) run on the M5 GPU at 100.5 ms per call; RapidOCR [23] reads the clock and deploy labels on a background worker; a template matcher reads the four hand slots (33.1 ms) and is re-run only when a slot crop changes (D-038); a Kalman tracker [24] follows units. The live-state builder turns detections into own and opponent elixir, hand, towers, the opponent's seen cards and each unit's time to reach a tower (ETA). Computing ETA from the card's speed and attack range instead of the raw track cut its median absolute error from 1.49 s to 0.30 s and raised the share within 1 s from 34% to 80% (384 frames, 11 arrivals, one video). The same builder produced the dense training data (D-040), so the play/wait model sees the same features live and offline.

TABLE I
Data and knowledge stores (train / val / test where split).

Pro games / videos (one creator)	197 / 33
Videos by split	23 / 5 / 5
Games by split	142 / 27 / 28
States by split	7,693 / 1,574 / 1,656
4-way ranker lines	3,641 / 761 / 766
Dense play/wait lines (one per second)	29,036 / 5,560 / 5,279
Pro-advice documents	3,654
KG v1: nodes / edges / relations	590 / 12,030 / 12
KG v2: nodes / edges / relations	840 / 27,112 / 19
lane-simulation edges (STOPS, KILLS, PULLS)	8,951
mined answer events → answer rules	3,620 → 287
situations / sequences / per-level stat rows	234 / 450 / 1,734

C. Data pipeline

A scraper first built a knowledge base of card, deck and match pages from the Clash Royale wiki. Then 197 games from 33 videos of one professional creator were perceived and cleaned into 10,923 states (6,901 plays, 3,539 key moments, 483 periodic samples). Bad fields are repaired and flagged, never deleted (D-011); for example, 2,165 states have an unknown placement. Splits are by video (SHA-1 order, 70/15/15), so no game appears in two splits, the contamination concern raised for RETRO [25]. A second pass re-perceived all 33 videos at one state per second (about 17 h on the M5) for play/wait labels: does the pro deploy within the next second? Table I gives the sizes.

D. Knowledge graph

KG v1 (590 nodes, 12,030 edges, 12 relations) holds wiki COUNTERS and SYNERGY relations, deck membership and 129 mined ANSWERED_BY edges. KG v2 (D-048) adds a game model: per-level stats for 126 cards (1,734 rows, opponents assumed at level 15), 8,951 STOPS, KILLS and PULLS edges computed by the lane simulator, 3,620 mined answer events (the opponent plays X; the pro's first defensive play within 8 s is Y) aggregated into 287 rules with held rates, placement offsets and timing, 234 situation nodes, 450 PLAYED_AFTER sequences, deck co-occurrence, and role-similarity transfer for our cards that have no pro data. An opponent-deck model completes the hidden deck from the cards seen. Live, the graph supplies the engine's answer prior, pro placement offsets (when $n \geq 5$), an elixir reserve for predicted air cards, a counter channel for candidates, and up to three `KG:` lines in d1's context, e.g. `KG: archers L15 x2 445hp 164/hit; answers mini-p-e-k-k-a (role of elite-barbarians, n=4), tesla (role of furnace, n=6).`

E. Retrieval

Each state is a 644-d structured vector (3 scalars, 5 phase values, five 120-d card blocks for hand, deck, opponent deck and recent plays of each side, 6 tower, 24 field and 6 threat values) plus a text rendering indexed by SQLite FTS5 BM25 [26]. Exact dot-product kNN and BM25 lists are fused by reciprocal rank fusion with $k = 60$ [27], and the neighbours vote for the plays they contain. The vector is hand-built, not a learned bi-encoder [28]. A 384-d bge-small text channel [29] and an HNSW index [30] were evaluated but are not on the live path (Sec. V-C).

F. Tactical engine

The engine (1,750 lines; roles for 122 cards) lists enemy pushes, estimates each push's tower damage with a lane simulation over per-level stats, and proposes placed answers with reasons. For example, a lone level-15 Balloon deals 3,162 tower HP unanswered, 351 with a Musketeer behind the tower, and 0 with a Tesla pull. It values spells by kills (a Zap needs at least two kills and value 2.5), leads spells by their flight time, keeps buildings inside a pull box, and tracks elixir trades and offence windows. Its analysis is written as `tactics:` lines and per-option notes, which is how d1 reads it (D-042).

G. When to act

A reactive policy (modes defend, leak, punish, counter, spell, save and none; D-041) holds by default and filters candidates by mode, e.g. defend keeps the threat's lane on our half. A LightGBM [31] model on 29 features predicts whether a pro would deploy within 2 s (threshold 0.47, 0.05 ms per call). In the live *fast* policy the rules override the model in defend, counter, spell and leak, and save and none always hold. This gate is our adaptive trigger [19]: it decides when the LLM is consulted at all.

H. Candidates and the d1 ranker

The generator returns at most four plays: up to two engine moves, then retrieval votes, KG v1 counter scores (weight 0.5) and legal fallbacks; the policy, the read hand and a hold rule for fallbacks below 9.5 elixir filter them. d1 is Liquid AI's d1-omni-600M [32], a 16-layer transformer trunk with a head that emits one logit per option marker, so one forward pass scores all options (setwise ranking [13]). Zero-shot it is at chance (Sec. V-B), so we fine-tuned the full text path (313,623,297 parameters, fp32 on MPS) with a softmax cross-entropy over the four options: `d1-ft-joint` on states alone (3 epochs, 13,241 s), then `d1-ft-joint-tac` on states with tactics lines and option notes (2 epochs, 4,554 s, peak 22.75 GB), the live model. Live, d1 takes 92.7/174.7 ms (p50/p95) and is called only when two or more candidates survive, which happened on 119 of 279 executed plays in the three latest runs.

The option-set shortcut (D-037). Our first training set built three negatives around each pro play: a neighbour's play, the same card elsewhere, and another card at the same spot. The positive was then always the option that shares its card with one option and its placement with another, and a rule that reads only the four option keys scored 0.848/0.853/0.852 on train/val/test. A model trained on it would learn the construction, not the game. We rebuilt every example as a 2×2 grid {pro card, second possible card} $\times$ {pro placement, second legal placement}: each option shares its card with exactly one other option and its placement with exactly one other, so the key-only rule scores 0.25 by construction while the negatives stay hard, in the spirit of RAFT's distractors [33]. A second, smaller tell (a fixed lane order; lane-only rule 0.31) was removed by randomising the second placement (lane-only rule now 0.272 on test).

I. Executor, safety and demo tooling

The executor maps a tile to screen points through the calibration and sends two Quartz clicks (card slot, then tile) 50 ms apart; the execute stage takes 183.3 ms at the median. It tracks own elixir after taps, detects failed taps and cools a failed card down for 2 s. Two independent switches gate every tap, the `--live` flag and `CLASHBOT_ALLOW_LIVE=1`, checked at construction and before every action (D-031). The demo launcher loads in about 10 s, watches the mirrored screen until stopped, and taps only in battles whose opponent banner reads "Trainer"; an unreadable banner counts as not Training Camp (D-050). It read 411 of 412 recorded banners and 5 of 5 battles correctly. The dashboard (Fig. 2) replays or follows a run: a timeline with hold reasons, the decision frame, candidates with d1 probabilities and sources, the ten retrieved pro states, the KG subgraph of the counters used, and the KG and tactics lines given to d1; logging them costs about 0.03 ms per decision.

IV. Course Concepts in the System

Table II maps each course block to what is built, where it runs and the number that tests it. Only built components are listed; techniques we cite but did not build (e.g. Matryoshka

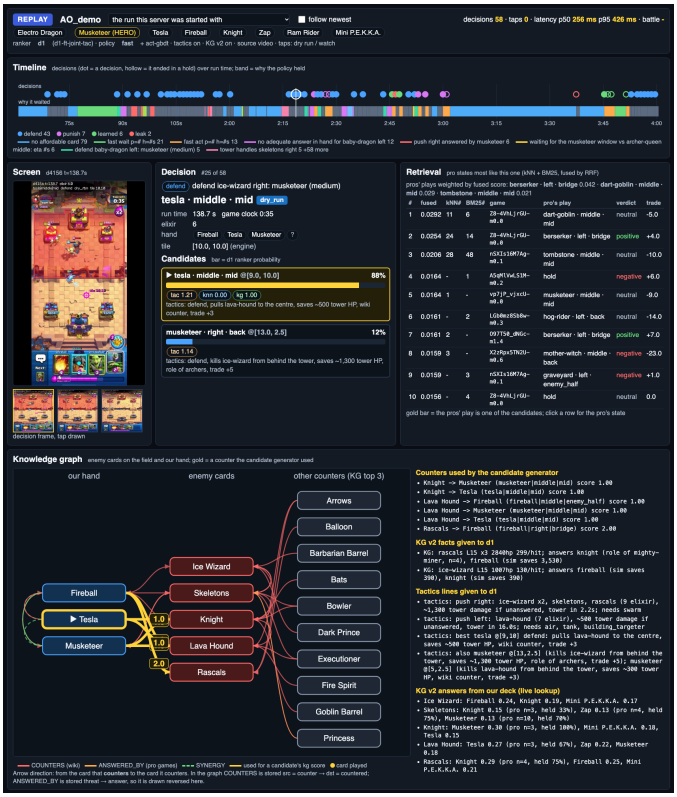


Fig. 2. Decision dashboard on a video dry run. Top: timeline of decisions and hold reasons. Middle: decision frame; two candidates with d1 probabilities (Tesla 88%) and the engine, kNN and KG scores behind them; the ten pro states retrieved by kNN+BM25 with RRF. Bottom: KG subgraph (our hand, enemy cards, counters used) and the `KG:` and `tactics:` lines d1 read.

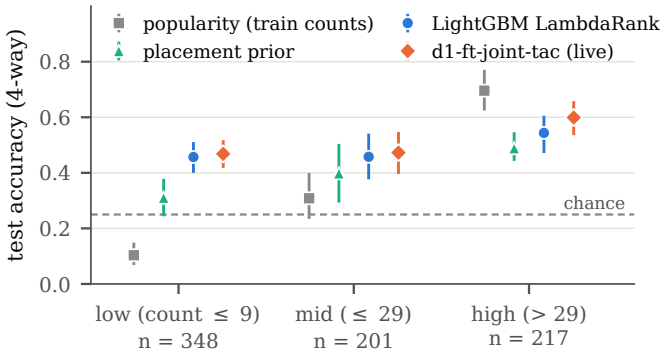


Fig. 3. Test accuracy by training frequency of the gold key (terciles; 95% cluster-bootstrap CIs). Popularity wins on head keys and fails on the tail; the fine-tuned LLM and LightGBM hold up on rare keys. Data: `stats_audit_v1.json`.

embeddings, ROME/MEMIT, Reflexion) are left out. Three course lessons recur in the results: tuned simple baselines are hard to beat [34]; the evaluation protocol matters as much as the model (D-037, the equal-budget KGE audit [35]); and knowledge in the context helps only when the model is trained to use it (Sec. V-B).

V. Experiments

A. Setup

Offline numbers come from the frozen decision code (D-051, commit `66917e7`); outputs that predated code changes were re-run. Ranker evaluations use 761 val and 766 test lines from 26 and 27 held-out games. CIs are percentile intervals from 10,000 cluster-bootstrap resamples of whole games (seed 20261107), paired across models; we also give exact McNemar p, which treats lines as independent and is therefore the optimistic test. Thresholds, ensemble weights and feature-ranker configurations are chosen on val, so test is the clean column.

B. Ranking the pro's play

Table III gives the main result. (1) Zero-shot d1 is at chance (0.258/0.221): the 600M model has no usable Clash Royale knowledge, the parametric-memory limit discussed in [2], [41]. (2) Fine-tuning lifts it to 0.547/0.507, 12.3 points above the placement prior on test (CI [+8.0, +16.7], $p < 0.001$). (3) The tuned feature ranker matches the LLM: d1-ft-joint-tac minus LightGBM is +2.5 [−1.4, +5.9] (McNemar $p = 0.263$), and against LightGBM's seed 2 it is −0.3 [−4.9, +4.0]. LightGBM's top features by gain are the option key's popularity (0.178) and $p(\text{zone} \mid \text{card})$ (0.124), the same priors as the simple baselines. (4) Training on the tactics notes is what makes them useful. Given the same notes, the model trained with them beats the model trained without them by +4.6 [+1.0, +8.4] ($p = 0.006$), and a model that never saw notes is hurt by them (0.461 vs 0.483 on test). The gain over the no-notes model on plain lines, +2.3 [−0.9, +5.3], is not significant; on the lines it was trained on it was +1.6/+1.4 points. (5) Retraining with `KG:` lines did not help (−2.2 [−4.7, +0.5] against d1-ft-joint-tac): validation loss rose from 1.1916 to 1.6708 in the second epoch, and the KG facts on training lines are mined from the same pro games, a leak we have not removed.

Fig. 3 splits test accuracy by how often the gold key occurs in training. Popularity alone is the best model on head keys (0.696) and the worst on the tail (0.103); d1-ft-joint-tac and LightGBM stay level across the two lower terciles (0.468/0.473 and 0.457/0.458), so both generalise beyond memorised keys. A share of 0.14 of test gold keys never occurs in training.

d1 is well calibrated (test ECE 0.048 over 15 bins; Brier 0.616), so its probability can drive abstention. Holding when the top probability is below 0.483 (chosen on val) answers 0.402 of test lines at 0.630 accuracy and raises the +1/0/−1 utility of CRAG-style scoring [42] from 0.013 [−0.056, 0.079] to 0.104 [0.060, 0.147] (Fig. 4b). An ensemble of d1 and LightGBM (d1 weight 0.35, fitted on val) scores 0.505 [0.472, 0.535] on test, no better than d1 alone (−0.1 points); with LightGBM's seed 1 the ensemble gains +3.9 [+0.9, +7.1], so the effect depends on the seed. A rule arbiter that narrows d1's options when d1 is unsure also gives no gain (test 0.506 for d1 alone, 0.509 with the arbiter), so it stays off.

TABLE II

COURSE TECHNIQUES BUILT INTO CLASHBOT. "LIVE" = ON THE DEFAULT LIVE PATH; "OFF" = OFFLINE EVALUATION OR A FLAG ONLY. TEST SPLIT UNLESS NOTED.

Block	Technique	Where in CLASHBOT	Evidence	Use
Retrieval L8–10 [36]–[38]	BM25 [26] + RRF (k=60) [27]	FTS5 text channel fused with structured kNN	hit@10 0.160 fused vs 0.136 kNN alone	live
	HNSW [30] vs exact search	index over 10,923 state vectors	ef 40: recall@10 0.978 at 0.31 ms vs 1.51 ms	off
	Pre- vs post-filtering	deck and phase filters	deck pool empty for 88.9% of queries; post@10 empty 50%/87%	off
	Setwise LLM reranking [13], [14]	d1 scores all ($\leq$4) options in one pass	0.507 fine-tuned vs 0.221 zero-shot (chance 0.25)	live
Dense repr. L2–4 [28], [39]	Fixed-length vectors; small text embedder [29]	644-d hand-built vector; 384-d bge-small channel	text channel adds $\sim$20 ms, hit@10 0.143: not integrated	live / off
RAG L11–12 [2], [40]	Parametric-memory limit [41]; retrieved context	engine, retrieval and KG notes in d1's context	zero-shot d1 at chance; accuracy vs key popularity (Fig. 3)	live
	Distractor training (RAFT) [33]	1 gold + 3 hard options on a 2×2 grid	option-key shortcut 0.852 $\rightarrow$ 0.250	train
	Graph facts in context (GraphRAG) [18]	up to 3 `KG:` lines per decision	retrain on them: $-$2.2 pts [$-$4.7, +0.5]	live
	Adaptive trigger [19], [20]	play/wait gate decides when d1 is consulted	LightGBM bal. acc. 0.607 vs rules 0.509 (h=2 s)	live
	Abstention scoring (CRAG) [42]	hold when d1's top probability $<$ 0.483	utility 0.104 vs 0.013 answering all	off
Rec. systems L13–14 [3], [43]	LM as recommender, full fine-tune [16], [17]	d1-omni-600M, 313.6M trainable parameters	val/test 0.547/0.507	live
	Feature rankers [44], [45]; tuned baselines [34]	logreg, LambdaRank, DCN-V2 on 783 features	d1 $-$ LightGBM: +2.5 pts [$-$1.4, +5.9]	off
	Sequential rec. (SASRec) [46]	opponent next-card prediction	acc@1 0.222 vs copy-frequency 0.243	off
KG L5–7 [47]–[49]	Typed KG feeding decisions	v1 590/12,030; v2 840/27,112, 19 relations	opp.-deck recall 0.234 vs popularity 0.130 (k=1)	live
	KGE [50]–[53], audit protocol [35], [54]	held-out COUNTERS triples, PyKEEN [55]	RotatE MRR 0.159 vs popularity 0.110	off
	Rule mining with support [56], [57]	287 "threat $\rightarrow$ pro answer" rules (8 s window)	hit@1 0.154 vs popularity 0.169, random 0.128	live
	Temporal copy mechanism [58]	copy-frequency next card; PLAYED_AFTER edges	acc@1 0.243, beats SASRec	off
	Neuro-symbolic hard constraints (L7)	legality filter, safety rules, lane simulation	pros' own play legal on 0.542–0.597 of states	live
Agents L15–16 [4], [59]	Agent with tools [10]; one decision LLM	engine, retrieval, KG propose, d1 decides (D-045)	13 runs, 931 logged decisions, 746 taps	live
	Guarded execution; trajectory-level evaluation [60]	two-switch guard, banner gate, per-decision logs	taps landed 95/100; win rate [TBD: k/10]	live
Knowledge upd. L17–18 [61], [62]	Editing criteria [63]; non-parametric edit [64]	patch-versioned fact store	reliability 1.000; portability 1.000 vs 0.067	off
	Knowledge conflicts [65], [66]	down-weight pre-patch hits; corrupted notes (E2)	0.124 $\rightarrow$ 0.002; E2 follow-rate [TBD]	off

TABLE III

4-WAY PICK-THE-PRO-PLAY ACCURACY (CHANCE 0.25) WITH 95% CLUSTER-BOOTSTRAP CIS. "NOTES" = TACTICS LINES AND PER-OPTION NOTES.

Model	Val	Test
Leak check and simple baselines		
option-key rule, old options / 2×2 grid	0.853 / 0.250	0.852 / 0.250
random	0.230 [0.207, 0.254]	0.252 [0.218, 0.284]
popularity of the key	0.468 [0.412, 0.516]	0.325 [0.280, 0.374]
placement prior	0.493 [0.430, 0.547]	0.384 [0.337, 0.431]
Feature rankers, 783 features (seed 0)		
logistic regression	0.543 [0.492, 0.586]	0.470 [0.423, 0.512]
LightGBM LambdaRank	0.565 [0.520, 0.604]	0.482 [0.448, 0.515]
mean $\pm$ sd over 3 seeds	0.5484 $\pm$ 0.0140	0.4913 $\pm$ 0.0126
DCN-V2	0.537 [0.493, 0.576]	0.403 [0.366, 0.439]
d1-omni-600M decision LLM		
zero-shot	0.258 [0.226, 0.287]	0.221 [0.191, 0.252]
ft-joint, plain lines	0.524 [0.483, 0.561]	0.483 [0.442, 0.522]
ft-joint, given notes it never saw	0.477 [0.427, 0.523]	0.461 [0.418, 0.506]
ft-joint-tac (trained on notes; live)	**0.547** [0.503, 0.584]	**0.507** [0.472, 0.540]
ft-joint-kg (also trained on `KG:` lines)	0.530 [0.480, 0.572]	0.484 [0.450, 0.518]

TABLE IV

RETRIEVAL CHANNELS (VAL N = 916 / TEST N = 902 QUERIES, TRAIN POOL, DEPTH 50). HIT@10: THE PRO'S EXACT PLAY IS AMONG THE TOP-10 NEIGHBOURS' PLAYS.

List	hit@10 val / test	card hit@10 val / test	ms p50 (test)
BM25 (FTS5)	0.142 / 0.159	0.396 / 0.483	25.8
structured kNN (644-d)	0.166 / 0.136	0.434 / 0.412	2.1
text (bge-small, 384-d)	0.132 / 0.143	0.397 / 0.436	22.2
struct + BM25 (live)	0.138 / 0.160	0.434 / 0.487	28.0
all three, RRF	0.156 / 0.170	0.461 / 0.514	49.6

C. Retrieval

Table IV: the channels are within noise of each other (test hit@10 0.136–0.170), and BM25 beats the structured channel on test (0.159 vs 0.136) but not on val. The text channel adds about 20 ms per query for no clear gain, so the live list is structured plus BM25. Retrieval alone is a weak candidate source: at the exact-key level, kNN voting reaches recall@10 0.3172 on test against 0.4021 for popularity, and wins only at the card level (0.6514 vs 0.6070). HNSW at ef 40 keeps recall@10 at 0.978 in 0.31 ms against 1.51 ms for exact search (Fig. 4a); at 10.9k vectors exact search is affordable, so the live path uses it. Filtering shows the pre- versus post-filter trade-off from the indexing lecture [37]: a deck pre-filter leaves an empty pool for 88.9% of queries (their decks never occur in training), and post-filtering the top 10 returns nothing for 50.0% (deck) and 87.3% (phase) of queries; post-filtering the top 200 brings this down to 0.5% and 0.1%.

D. Knowledge graph: embeddings and mined rules

Link prediction (Table Va). We hold out wiki COUNTERS triples (3,043/380/381) and rank 126 candidate cards with filtered, tie-aware ranks. All four PyKEEN [55] models get the same grid (dimension 32 or 64, learning rate 0.01 or 0.001), chosen on val MRR, and three seeds, following the course's evaluation-audit protocol [35], [47], [54]. RotatE [53] is best (MRR 0.1588, H@10 0.3259) and beats popularity (0.1103);

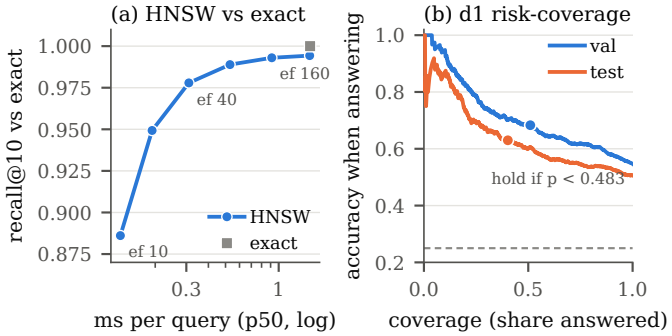


Fig. 4. (a) HNSW recall@10 against exact kNN versus query time (10,923 vectors, k=10). (b) Risk–coverage of d1-ft-joint-tac: accuracy on the lines it answers when it holds below a probability threshold; dots mark the threshold 0.483 chosen on val. Data: `retrieval_ablation_v1.json`, `stats_audit_v1_risk_coverage.csv`.

TABLE V

(A) HELD-OUT COUNTERS LINK PREDICTION, TEST (762 QUERIES, FILTERED, 3 SEEDS). (B) RANKING THE PRO'S OWN DECK FOR A HELD-OUT THREAT (E6); RULES MINED FROM TRAINING GAMES ONLY.

(a) KG embeddings

Method	MRR	H@1	H@10
random	0.0223	0.0113	0.1129
popularity	0.1103	0.0381	0.2385
TransE	0.0819 ± 0.0046	0.0000	0.2380
DistMult	0.0907 ± 0.0083	0.0232	0.1986
ComplEx	0.1162 ± 0.0074	0.0367	0.2616
RotatE	**0.1588** ± 0.0020	0.0713	0.3259

(b) Mined answer rules (609 val / 433 test events)

Score	hit@1 val	hit@1 test	MRR test
full `KG2.answers_for`	0.182	0.154	0.374
pro-mined rules only	0.191	0.129	0.350
wiki COUNTERS only	0.117	0.141	0.359
train popularity of the answer card	0.182	0.169	0.384
random card in deck (expected)	0.126	0.128	0.345

TransE (0.0819) and DistMult (0.0907) do not. DistMult is symmetric by construction (asymmetry score 0.0), while only 940 of 3,804 counter pairs have a reverse edge.

Mined answer rules (E6, Table Vb). For each held-out event we rank the pro's own deck (7.96/7.83 cards on average) with rules mined from training games (support ≥ 3, 8 s window). Pooled over val and test, the full score beats random (+0.044 hit@1, CI [+0.020, +0.068]) and the wiki counters (+0.043 [+0.011, +0.074]) but not the training popularity of the answer card (−0.006 [−0.035, +0.020]); on test the pro-mined source alone falls below popularity (−0.040 hit@1, $p = 0.022$). The score barely predicts whether the answer held the tower (AUROC 0.543), few rules have large support (best support 10 or more for 63 of 1,042 events), and the 8 s window is the best of 2, 4 and 8 s on val (MRR 0.409). The mined rules are a weak prior, in line with the course's caution that rule confidence needs held-out checking [49], [56].

E. Context conflict: does d1 follow wrong notes? (E2)

Following ClashEval and the knowledge-conflict lecture [61], [65], [66], we corrupt the context of the `joint_tac` lines without changing the gold label: C1 removes notes and the `tactics:/KG:` lines, C2 shuffles the notes across options, C3 swaps the gold option's note with a distractor's (relevant but misleading), and C4 transplants the tactics/KG block of another line (irrelevant). Follow-rate is the share of lines whose pick moves to the option the corrupted note favours. Table VI: [TBD from context_conflict_v1: which conditions hurt and by how much].

TABLE VI

CONTEXT CONFLICT ON D1-FT-JOINT-TAC (E2), TEST LINES, 95% CLUSTER-BOOTSTRAP CI.

Condition	Accuracy [CI]	Δ vs C0 (McNemar p)	Follow-rate
C0 as exported	[TBD]	–	–
C1 notes removed	[acc]	[Δ (p)]	–
C2 notes shuffled	[acc]	[Δ (p)]	[rate]
C3 gold note swapped	[acc]	[Δ (p)]	[rate]
C4 block transplanted	[acc]	[Δ (p)]	[rate]

TABLE VII

PLAY/WAIT TIMING ON DENSE TEST SECONDS (N = 5,279): BALANCED ACCURACY FOR "THE PRO DEPLOYS WITHIN h".

Policy	h = 1 s	h = 2 s	h = 3 s
hand-written rules	0.506	0.509	0.510
LightGBM, 29 features	0.594	**0.607**	0.613
AUC	0.632	0.659	0.672
rules + LightGBM (live *fast*)	0.571	0.584	0.593
Balanced act-head subset, h = 1 s (n = 3,221)			
d1 act head (fine-tuned LLM)	0.558		
LightGBM, same subset	0.598		

F. When to act

Table VII: LightGBM reaches balanced accuracy 0.607 (AUC 0.659) at $h = 2$ s against 0.509 for the hand-written rules, and the live combination, in which rules override the model in four modes, sits between them (0.584). On the balanced subset used to train a d1 act head (top 8 layers, 168.9M trainable parameters), LightGBM scores 0.598 against the head's 0.558, at 0.05 ms instead of about 70 ms per call: a second case where the tuned simple model beats the LLM. Offline on pro states, the reactive rules act on 0.220 of moments against a pro play rate of 0.941 on those play-biased samples (balanced accuracy 0.545).

G. Next card, opponent deck and knowledge editing

Opponent next-card prediction is a sequential recommendation task [43], [46]. SASRec (3 seeds) reaches acc@1 0.129/0.222 on val/test and loses to a copy-frequency rule (0.207/0.243), the copy intuition of CyGNet [48], [58]; popularity is near zero (0.000/0.002) because opponents vary, and the ceiling set by cards already seen is 0.678/0.644. The KG-based opponent-deck model recovers hidden cards with recall 0.234/0.291/0.327 after 1/2/3 seen cards, against 0.130/0.124/0.118 for popularity (leave-one-game-out, 155 games). For knowledge editing [61], [63] we apply a synthetic +1-elixir patch to five cards in a patch-versioned fact store, the non-parametric route of SERAC [64]: reliability (n=15), generalisation (n=465) and portability (n=465) are 1.000,

TABLE VIII
BOT VS PRO ON HELD-OUT PRO VIDEOS, OPEN LOOP ([TBD: N] GAMES). MATCHING WINDOW ±2 S; CIS BY GAME BOOTSTRAP.

Measure	Value [95% CI]
pro plays matched by a bot act (recall)	[r]
bot acts matched by a pro play (precision)	[p]
acts per minute, bot / pro	[a / b]
same card on matched pairs	[c]
baseline: random card in the read hand	[c0]
baseline: placement-prior key	[c1]
same lane on matched pairs	[l]
top hold reason on missed pro plays	[reason]

TABLE IX
(A) DECISION LATENCY BY STAGE, MS P50 / P95. (B) LIVE RECORD IN TRAINING CAMP.

(a) Stage	live run 111554	video dry run
capture	21.2 / 32.1	–
perception (detector, OCR, hand)	102.4 / 230.0	140 / 196
live state	0.13 / 0.33	–
trigger (rules + LightGBM)	1.5 / 8.7	–
candidates (engine, retrieval, KG)	26.1 / 56.9	45 / 79
rank (d1; 0 when one candidate)	0.0 / 136.7	84 / 174
execute (two clicks)	183.3 / 188.1	dry run
total	**338.9 / 571.8**	248 / 402
total, 3 video repetitions (mean ± sd)	–	[TBD: latency_v2]

(b) Live record	
recorded runs 11-03 to 11-07	13 (12 live), 931 decisions, 746 taps
taps landed, 3 latest runs	95/100, 75/84, 78/95
run 013441 (3 battles)	tower taken; 3-crown loss; **win in overtime**
trial battles / wins	[TBD: 10] / [TBD: k]
win rate, exact 95% CI	[TBD: k/10 [lo, hi]]
crowns for / against per battle	[TBD: x / y]
decisions / taps per battle	[TBD: d / t]

against 0.000, 0.000 and 0.067 without the edit; locality is 1.000 on 2,500 cards and 1,500 states; versioning and rollback pass; and re-weighting cuts the share of retrieved hits that use an edited card from 0.124 to 0.002. The store is deterministic, so these scores show that the mechanism works, not that it generalises to a real balance patch [67].

H. Bot versus pro on the pros' own videos

We run the full live stack open-loop on test-split videos with the pro's deck (dry-run executor, live flags) and match the bot's would-be taps to the pro's plays by time. Open loop means the bot's actions do not change the video, so this measures what the bot would do in the pro's situation, not whether it would win. Table VIII: [TBD from vs_pro_v1: n test games, minutes; recall, precision, same-card rate vs baselines].

I. Live play and comparison with prior bots

Thirteen recorded runs between 2026-11-03 and 2026-11-07 (12 live, one dry) logged 931 decisions, of which 746 sent a tap. In the latest live run (Table IXa) perception (102.4 ms p50) and the two-click execute stage (183.3 ms) dominate; d1 adds nothing at the median, because most decisions have a single candidate, and 136.7 ms at p95. The 300 ms p95 target was missed in every run. Over all 931 decisions the total is 347/613 ms. Taps landed on 95/100, 75/84 and 78/95 in the three latest runs; in the last one, failures concentrate on Tesla

TABLE X
PRIOR CLASH ROYALE AGENTS AND CLASHBOT. R = REPORTED BY THE AUTHORS, M = MEASURED BY US. PROTOCOLS DIFFER; SEE TEXT.

	KataCR [1]	Claude Royale [9]	CLASHBOT (ours)
Interface	phone video stream, ADB taps (Linux)	BlueStacks on macOS	iPhone Mirroring on macOS, Quartz clicks
Perception	YOLOv8-l×2 (mAP 67.4), OCR, ResNet classifiers	Claude vision on screenshots	KataCR's YOLOv8 weights, OCR, Kalman tracker
Decision maker	offline-RL StARformer	Claude Haiku, 3 player agents	gate + engine + retrieval + KG; fine-tuned 600M LLM picks
Knowledge used at inference	none (in weights)	hand-written prompts and notes	10,923 pro states, KG v1/v2, tactics engine
Training data	105 episodes, 113,981 frames, one player vs AI	none (prompting)	197 pro games, 33 videos
Latency	120 ms decision + 240 ms fusion (R)	∼7 s per agent (R)	339 / 572 ms p50 / p95, capture to click (M)
Offline decision metric	none	card choice 11/25 on labelled screenshots (R)	4-way test accuracy 0.507 (M)
Opponent	built-in AI, Training Camp	human ladder	built-in AI, Training Camp
Win rate vs built-in AI	1/20 = 5% headline; 0–10% over 9 models (R)	not reported	[TBD: k/10]; first win on 2026-11-07 (M)
95% CI	0.1–24.9%	–	[TBD: lo–hi]

(11 of 19 landed) and Fireball (3 of 7). In run 20261107-013441 the bot took a tower in game 1, lost game 2 by three crowns and won game 3 in overtime, its first win. An earlier controlled pair showed the tactics context at work: without it the bot damaged one enemy tower by 830 HP in a 1 min 30 s loss; with it, both towers by 830 HP in a 2 min 10 s loss. *Trial.* [TBD: n = 10 Training Camp battles through the launcher on the frozen code: k wins, exact 95% CI, crowns, tap landing; one-sided Fisher test against KataCR's 1/20: p]. With ten battles, at least four wins are needed for that test to reach $p < 0.05$ ($p = 0.031$ at 4/10, 0.095 at 3/10).

Table X compares the three agents. Only KataCR and CLASHBOT share a "versus built-in AI" row, and even there protocols differ: KataCR played the 2024 game (Season 59) and today's Training Camp opponent may not be its "8000-rated" AI, the decks differ, and its reported row is the best of ten epochs selected on the same 20 games, an optimistic estimate. Latency definitions also differ: about 0.36 s for KataCR if its two stages run in series (actuation excluded), 0.34 s p50 for ours (actuation included), and about 7 s per agent for Claude Royale. We reuse KataCR's detector and do not claim a better one.

VI. DISCUSSION AND LIMITATIONS

Perception noise dominates. On one re-perceived pro game, the live state matched the dataset on phase 0.62, towers 0.77, elixir 0.55 and hand slots 0.57 of records, units Jaccard 0.34, median feature cosine 0.6714 (measured before later perception fixes); the pro's own move is legal under our reading on only 0.542–0.597 of states; in an early live run 14 of 27 decisions were skipped because the hand read as unknown. Better models cannot recover from a wrong board. *The engine*

is weak alone but useful as context. As a ranker, the tactics engine is near chance against the pro (4-way 0.293/0.265 val/test; top-1 card 0.100 on test), yet training d1 on its notes is worth +4.6 points over the same notes unseen (Sec. V-B). *d1's live influence is limited.* d1 ranks only when two or more candidates survive: 119 of 279 executed plays. Its context at serve time also carries a `[plan]` line that no training line contains, a train/serve mismatch whose effect we have not measured. *Live weaknesses.* In the latest run about a third of match frames had no affordable card and 23 emergency plays were made at 2–6 elixir; in run 013441 the median response came 5.4 s after the opponent's play. *Small n and one source.* Every offline number rests on 26–27 held-out games (CIs about ± 3.5 points) from one creator, and three of our eight cards have no pro plays at all, so the engine and role transfer carry them. Video evaluation is open loop, the perception does not read enemy health bars (units are simulated at full HP), and the larger d1-2B fine-tune reached only 0.4192 on val and was never tested because its checkpoint failed to load.

VII. Conclusion

CLASHBOT shows that a non-embedded agent can play Clash Royale live on a laptop, with knowledge-graph, retrieval and simulation tools feeding a fine-tuned LLM that keeps the final say, at 339 ms from capture to click. It won a Training Camp battle, [TBD: and k of 10 in the trial]. The honest summary of the benchmarks is that fine-tuning turned a chance-level LLM into one that picks the pro's play half the time, but tuned simple models (LightGBM, placement prior, copy-frequency) match or beat the LLM and the course's more elaborate methods at most sub-tasks, and context helps only when the model is trained on it. The next steps follow from the limitations: perception accuracy against labelled frames, retraining d1 on leak-free KG lines and coach plans, and a larger live trial. Code, tests, benchmark files (`docs/data/BENCHMARKS.md`) and the decision log are in the project repository.

References

[1] T. Wu, L. Wan, Y. Wang, Q. Wan, and X. Lan, "Playing non-embedded card-based games with reinforcement learning," arXiv:2504.04783, 2025, also in ICIRA 2024, LNCS vol. 15206, pp. 255–269, doi 10.1007/978-981-96-0792-1_20. [Online]. Available: https://arxiv.org/abs/2504.04783

[2] S. N. Lim, "Retrieval-augmented generation I: Architectural formulations, joint pretraining & fine-tuning dynamics, contextual mechanics, and rigorous evaluation frameworks," Lecture 11, Topic 05, CAP 6938 AI Knowledge Systems, University of Central Florida, Oct. 2026, lecture slides, October 1, 2026.

[3] ——, "Recommendation I: Collaborative filtering, ranking losses, and offline evaluation," Lecture 13, Topic 06, CAP 6938 AI Knowledge Systems, University of Central Florida, Oct. 2026, lecture slides, October 8, 2026.

[4] ——, "AI agents I: Tools, reasoning loops, planning, and context management," Lecture 15, Topic 07, CAP 6938 AI Knowledge Systems, University of Central Florida, Oct. 2026, lecture slides, October 20, 2026.

[5] O. Vinyals, I. Babuschkin, W. M. Czarnecki *et al.*, "Grandmaster level in StarCraft II using multi-agent reinforcement learning," *Nature*, vol. 575, no. 7782, pp. 350–354, 2019.

[6] C. Berner, G. Brockman, B. Chan *et al.*, "Dota 2 with large scale deep reinforcement learning," arXiv:1912.06680, 2019.

[7] L. Chen, K. Lu, A. Rajeswaran, K. Lee, A. Grover, M. Laskin, P. Abbeel, A. Srinivas, and I. Mordatch, "Decision transformer: Reinforcement learning via sequence modeling," in *Advances in Neural Information Processing Systems 34 (NeurIPS)*, 2021.

[8] T. Wu, "KataCR: Non-embedded Clash Royale agent (code, detection dataset and YOLOv8 weights)," GitHub repository, https://github.com/wty-yy/katacr, 2024, commit 36ceb9f; MIT licence; detector weights v0.7.13; accessed 2026-11-07.

[9] E. Houseworth, "Claude Royale: AI agents playing Clash Royale autonomously," GitHub repository, https://github.com/houseworthe/claude-royale, 2025, created 2025-12-07; last commit 2026-01-25 (70ecc8c); MIT licence; accessed 2026-11-07.

[10] S. Yao, J. Zhao, D. Yu *et al.*, "ReAct: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations (ICLR)*, 2023, arXiv:2210.03629.

[11] N. Shinn, F. Cassano, A. Gopinath *et al.*, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023, arXiv:2303.11366.

[12] A. Zhou, K. Yan, M. Shlapentokh-Rothman *et al.*, "Language agent tree search unifies reasoning, acting, and planning in language models," in *International Conference on Machine Learning (ICML)*, 2024, arXiv:2310.04406.

[13] S. Zhuang, H. Zhuang, B. Koopman, and G. Zuccon, "A setwise approach for effective and highly efficient zero-shot ranking with large language models," in *Proceedings of the 47th International ACM SIGIR Conference*, 2024, arXiv:2310.09497.

[14] W. Sun, L. Yan, X. Ma, P. Ren, D. Yin, and Z. Ren, "Is ChatGPT good at search? investigating large language models as re-ranking agents," in *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023, arXiv:2304.09542.

[15] X. Yi, J. Yang, L. Hong, D. Z. Cheng, L. Heldt, A. Kumthekar, Z. Zhao, L. Wei, and E. Chi, "Sampling-bias-corrected neural modeling for large corpus item recommendations," in *ACM Conference on Recommender Systems (RecSys)*, 2019.

[16] K. Bao, J. Zhang, Y. Zhang, W. Wang, F. Feng, and X. He, "TALLRec: An effective and efficient tuning framework to align large language model with recommendation," in *ACM Conference on Recommender Systems (RecSys)*, 2023, arXiv:2305.00447.

[17] S. Geng, S. Liu, Z. Fu, Y. Ge, and Y. Zhang, "Recommendation as language processing (RLP): A unified pretrain, personalized prompt & predict paradigm (P5)," in *ACM Conference on Recommender Systems (RecSys)*, 2022, arXiv:2203.13366.

[18] D. Edge, H. Trinh, N. Cheng *et al.*, "From local to global: A graph RAG approach to query-focused summarization," *arXiv preprint arXiv:2404.16130*, 2024.

[19] Z. Jiang, F. F. Xu, L. Gao, Z. Sun, Q. Liu, J. Dwivedi-Yu, Y. Yang, J. Callan, and G. Neubig, "Active retrieval augmented generation," in *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023, arXiv:2305.06983.

[20] S. Jeong, J. Baek, S. Cho, S. J. Hwang, and J. C. Park, "Adaptive-RAG: Learning to adapt retrieval-augmented large language models through question complexity," in *Proceedings of the Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2024, arXiv:2403.14403.

[21] Qwen Team, "Qwen3 technical report," arXiv:2505.09388, 2025.

[22] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," https://github.com/ultralytics/ultralytics, 2023, version 8.0.0; AGPL-3.0.

[23] RapidAI, "RapidOCR: An OCR toolkit based on ONNXRuntime," GitHub repository, https://github.com/RapidAI/RapidOCR, 2021, package rapidocr_onnxruntime; accessed 2026-11-07.

[24] R. E. Kalman, "A new approach to linear filtering and prediction problems," *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, 1960.

[25] S. Borgeaud, A. Mensch, J. Hoffmann *et al.*, "Improving language models by retrieving from trillions of tokens," in *International Conference on Machine Learning (ICML)*, 2022, arXiv:2112.04426.

[26] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[27] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, "Reciprocal rank fusion outperforms Condorcet and individual rank learning methods," in *Proceedings of the 32nd International ACM SIGIR Conference*, 2009.

[28] S. N. Lim, "Dense knowledge representation I: Fixed-length vectors, the objectives that shape them, and the training choices that matter,"

Lecture 02, CAP 6938 AI Knowledge Systems, University of Central Florida, Sep. 2026, sep. 1, 2026.

[29] S. Xiao, Z. Liu, P. Zhang, and N. Muennighoff, "C-Pack: Packaged resources to advance general Chinese embedding," arXiv:2309.07597, 2023.

[30] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2020, arXiv:1603.09320.

[31] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 2017, pp. 3146–3154.

[32] Liquid AI, "d1-omni-600M," Hugging Face model card, https://huggingface.co/LiquidAI/d1-omni-600M, 2026, lFM Open License; accessed 2026-11-07.

[33] T. Zhang, S. G. Patil, N. Jain, S. Shen, M. Zaharia, I. Stoica, and J. E. Gonzalez, "RAFT: Adapting language model to domain specific RAG," *arXiv preprint arXiv:2403.10131*, 2024.

[34] S. Rendle, W. Krichene, L. Zhang, and J. Anderson, "Neural collaborative filtering vs. matrix factorization revisited," in *ACM Conference on Recommender Systems (RecSys)*, 2020, arXiv:2005.09683.

[35] D. Ruffinelli, S. Broscheit, and R. Gemulla, "You CAN teach an old dog new tricks! on training knowledge graph embeddings," in *ICLR*, 2020.

[36] S. N. Lim, "Retrieval I: Representation," Lecture 08, CAP 6938 AI Knowledge Systems, University of Central Florida, Sep. 2026, sep. 29, 2026.

[37] ——, "Retrieval II: Indexing and search," Lecture 09, CAP 6938 AI Knowledge Systems, University of Central Florida, Oct. 2026, oct. 1, 2026.

[38] ——, "Retrieval III: Ranking and generative retrieval," Lecture 10, CAP 6938 AI Knowledge Systems, University of Central Florida, Oct. 2026, oct. 6, 2026.

[39] ——, "Dense knowledge representation II: Unified multimodal embedders, and DINO self-supervised vision, v1 to v3," Lecture 03, CAP 6938 AI Knowledge Systems, University of Central Florida, Sep. 2026, sep. 3, 2026.

[40] ——, "Retrieval-augmented generation II: Dynamic retrieval triggers, reinforcement learning for autonomous search agents, and graph-structured indexing architectures," Lecture 12, Topic 05, CAP 6938 AI Knowledge Systems, University of Central Florida, Oct. 2026, lecture slides, October 6, 2026.

[41] A. Mallen, A. Asai, V. Zhong, R. Das, D. Khashabi, and H. Hajishirzi, "When not to trust language models: Investigating effectiveness of parametric and non-parametric memories," in *Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023, arXiv:2212.10511.

[42] X. Yang, K. Sun, H. Xin *et al.*, "CRAG: Comprehensive RAG benchmark," in *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2024, arXiv:2406.04744.

[43] S. N. Lim, "Recommendation II: Sequential and generative recommenders," Lecture 14, Topic 06, CAP 6938 AI Knowledge Systems, University of Central Florida, Oct. 2026, lecture slides, October 8, 2026.

[44] R. Wang, R. Shivanna, D. Cheng, S. Jain, D. Lin, L. Hong, and E. Chi, "DCN V2: Improved deep & cross network and practical lessons for web-scale learning to rank systems," in *Proceedings of the Web Conference (WWW)*, 2021, arXiv:2008.13535.

[45] C. J. C. Burges, "From RankNet to LambdaRank to LambdaMART: An overview," Microsoft Research, Tech. Rep. MSR-TR-2010-82, 2010.

[46] W.-C. Kang and J. McAuley, "Self-attentive sequential recommendation," in *IEEE International Conference on Data Mining (ICDM)*, 2018, arXiv:1808.09781.

[47] CAP 6938 Course Staff, "Lecture 5: Knowledge graphs," Course lecture slides, CAP 6938, University of Central Florida, 2026.

[48] ——, "Lecture 6: Temporal and visual knowledge graphs," Course lecture slides, CAP 6938, Topic 02C, University of Central Florida, 2026.

[49] ——, "Lecture 7: Symbolic and neuro-symbolic reasoning," Course lecture slides, CAP 6938, University of Central Florida, 2026.

[50] A. Bordes, N. Usunier, A. Garcia-Duran, J. Weston, and O. Yakhnenko, "Translating embeddings for modeling multi-relational data," in *NeurIPS*, 2013.

[51] B. Yang, W.-t. Yih, X. He, J. Gao, and L. Deng, "Embedding entities and relations for learning and inference in knowledge bases," in *ICLR*, 2015.

[52] T. Trouillon, J. Welbl, S. Riedel, É. Gaussier, and G. Bouchard, "Complex embeddings for simple link prediction," in *ICML*, 2016.

[53] Z. Sun, Z.-H. Deng, J.-Y. Nie, and J. Tang, "Rotate: Knowledge graph embedding by relational rotation in complex space," in *ICLR*, 2019.

[54] Z. Sun, S. Vashishth, S. Sanyal, P. Talukdar, and Y. Yang, "A re-evaluation of knowledge graph completion methods," in *ACL*, 2020.

[55] M. Ali, M. Berrendorf, C. T. Hoyt, L. Vermue, M. Galkin, S. Sharifzadeh, A. Fischer, V. Tresp, and J. Lehmann, "Bringing light into the dark: A large-scale evaluation of knowledge graph embedding models under a unified framework," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2021.

[56] J. Lajus, L. Galárraga, and F. Suchanek, "Fast and exact rule mining with AMIE 3," in *ESWC*, 2020.

[57] Y. Liu, Y. Ma, M. Hildebrandt, M. Joblin, and V. Tresp, "TLogic: Temporal logical rules for explainable link forecasting on temporal knowledge graphs," in *AAAI*, 2022.

[58] C. Zhu, M. Chen, C. Fan, G. Cheng, and Y. Zhang, "Learning from history: Modeling temporal knowledge graphs with sequential copy-generation networks," in *AAAI*, 2021.

[59] S. N. Lim, "AI agents II: Memory, long-horizon agents, and multi-agent systems," Lecture 16, Topic 07, CAP 6938 AI Knowledge Systems, University of Central Florida, Oct. 2026, lecture slides, October 22, 2026.

[60] S. Yao, N. Shinn, P. Razavi *et al.*, "τ-bench: A benchmark for tool-agent-user interaction in real-world domains," in *International Conference on Learning Representations (ICLR)*, 2025, arXiv:2406.12045.

[61] S. N. Lim, "Knowledge updating I: Model editing and knowledge conflicts," Lecture 17, Topic 08, CAP 6938 AI Knowledge Systems, University of Central Florida, Oct. 2026, lecture slides, October 27, 2026.

[62] ——, "Knowledge updating II: Continual learning and knowledge injection," Lecture 18, Topic 08, CAP 6938 AI Knowledge Systems, University of Central Florida, Oct. 2026, lecture slides, October 29, 2026.

[63] Y. Yao, P. Wang, B. Tian *et al.*, "Editing large language models: Problems, methods, and opportunities," in *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023, arXiv:2305.13172.

[64] E. Mitchell, C. Lin, A. Bosselut, C. D. Manning, and C. Finn, "Memory-based model editing at scale," in *International Conference on Machine Learning (ICML)*, 2022, arXiv:2206.06520.

[65] K. Wu, E. Wu, and J. Zou, "ClashEval: Quantifying the tug-of-war between an LLM's internal prior and external evidence," in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024, arXiv:2404.10198.

[66] J. Xie, K. Zhang, J. Chen, R. Lou, and Y. Su, "Adaptive chameleon or stubborn sloth: Revealing the behavior of large language models in knowledge conflicts," in *International Conference on Learning Representations (ICLR)*, 2024, arXiv:2305.13300.

[67] W. Yang, F. Sun, J. Tan *et al.*, "The mirage of model editing: Revisiting evaluation in the wild," in *Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025, arXiv:2502.11177.